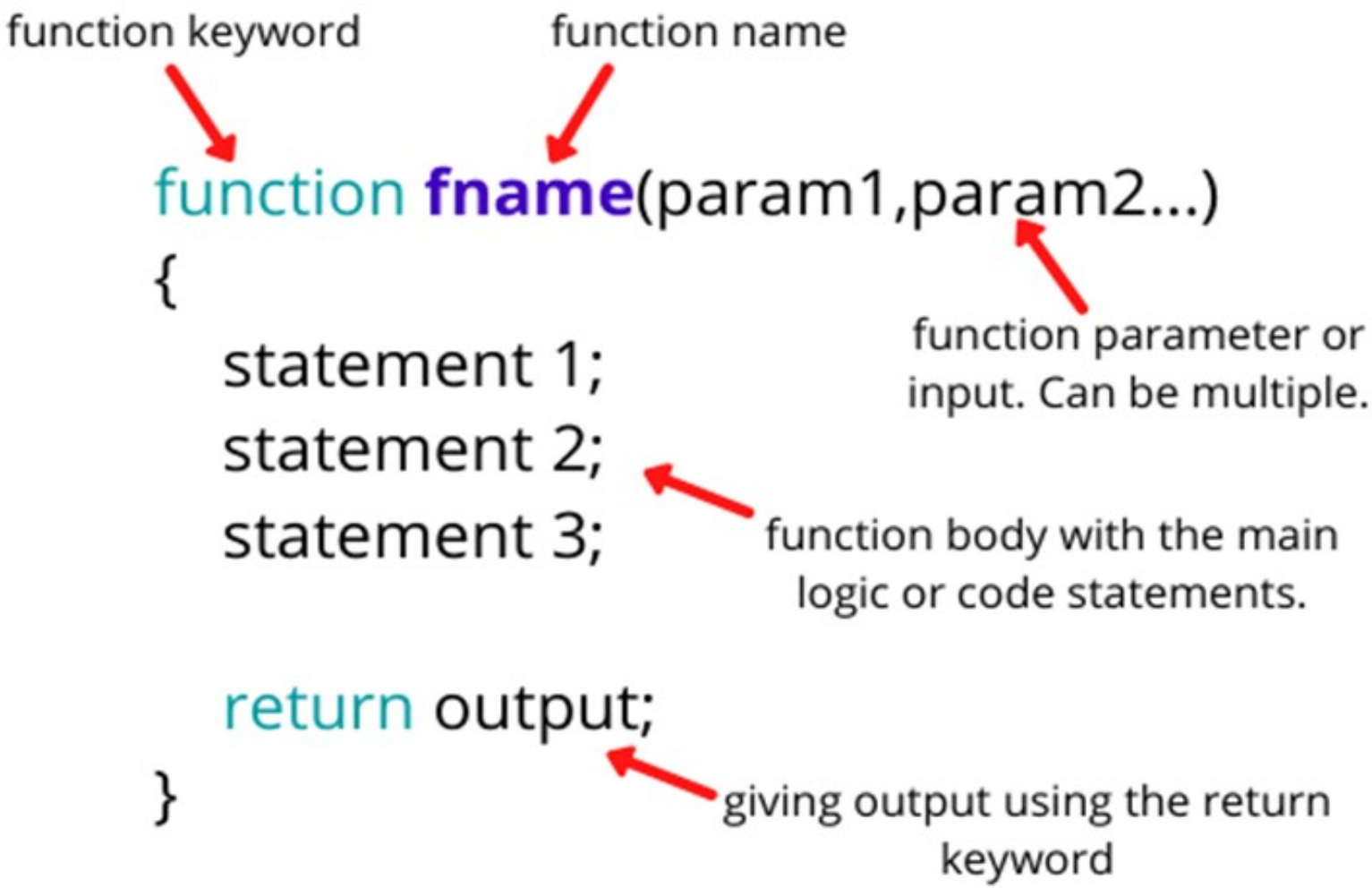
FUNCTIONS

FUNCTIONS

- In JavaScript, a function allows you to define a block of code, give it a name and then execute it as many times as you want.
- Simply its a block of code designed to perform a particular task.
- A JavaScript function is executed when "something" invokes it (calls it).

Function Definition

- A JavaScript function can be defined using function keyword.
- A JavaScript function is defined with the function keyword, followed by a name, followed by parentheses ().
- Function names can contain letters, digits, underscores, and dollar signs (same rules as variables).
- The parentheses may include parameter names separated by commas: (parameter1, parameter2, ...)
- The code to be executed, by the function, is placed inside curly brackets: {}


function keyword
function name
function fname(param1,param2...)
{
statement 1;
statement 2;
statement 3;
return output;
}
function parameter or input. Can be multiple.
function body with the main logic or code statements.
giving output using the return keyword

Function Invocation

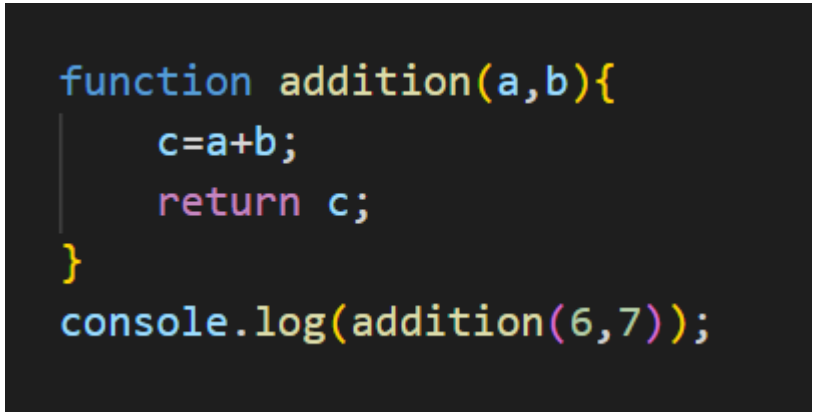
- The code inside the function will execute when "something" invokes (calls) the function:
- When an event occurs (when a user clicks a button)
- When it is invoked (called) from JavaScript code Automatically (self invoked)

//calling a function

```
<function-name>();
```

Function Return

- When JavaScript reaches a return statement, the function will stop executing.
- If the function was invoked from a statement, JavaScript will "return" to execute the code after the invoking statement.
- Functions often compute a return value. The return value is "returned" back to the "caller":
- Eg:



Creating and Calling a Function

Creating a Function

Syntax: -

```
function function_name( )
{
    Block of statement;
}
```

Body of Function

Calling a Function

Syntax: -

```
function_name( );
```

Ex: -

```
display( );
```

Function Syntax

//defining a function

function <function-name>()

{

    // code to be executed

};

```
function add(a,b){
    console.log(a+b);
}
add(4,5)



function addition(a,b){
    c=a+b;
    return c;
}
console.log(addition(6,7));
```

Arrow Functions

- Arrow functions, introduced in **ECMAScript 6 (ES6),** provide a more concise syntax for writing functions in JavaScript.
- They differ from traditional function expressions in several ways:

1.Syntax

Arrow Function:

```
const add = (a, b) => a + b;
```

Traditional Function:

```
function add(a, b) {
  return a + b;
}
```

If the function has only one expression, the return keyword and curly braces can be omitted.

```
() => expression

param => expression

(param) => expression

(param1, paramN) => expression

() => {
  statements
}

param => {
  statements
}

(param1, paramN) => {
  statements
}
```

```
// Traditional anonymous function
(function (a) {
  return a + 100;
});

// 1. Remove the word "function" and place arrow between the argument and opening body
brace
(a) => {
  return a + 100;
};

// 2. Remove the body braces and word "return" — the return is implied.
(a) => a + 100;

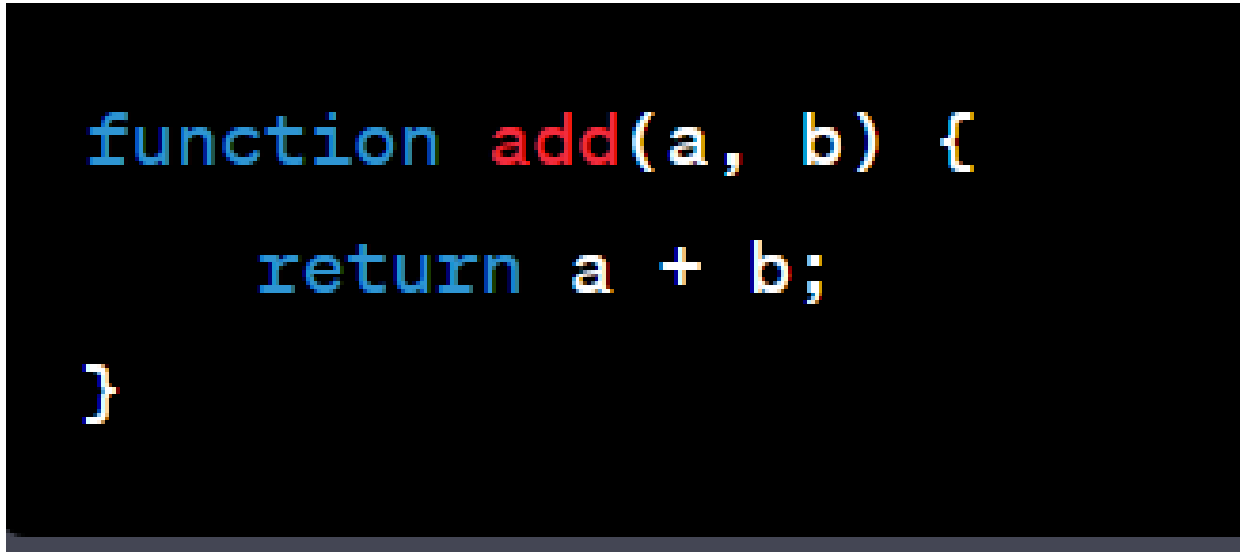
// 3. Remove the parameter parentheses
a => a + 100;
```

Types Of Functions

1. **Named function**
2. **Anonymous Functions**
3. **Recursive Functions**

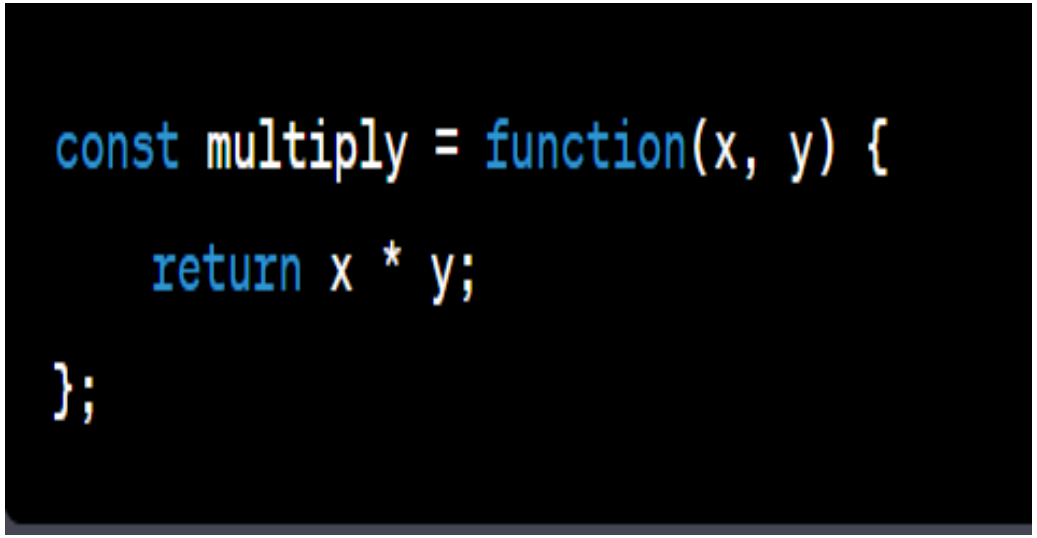
1 Named function

- These are functions defined with a name that can be called by that name.
- Named functions are useful if we need to call a function **many times** to pass **different values** to it or run it several times.



2 Anonymous Functions

- These are functions without a name and are often used as arguments to other functions or as immediately invoked function expressions (IIFE).



3 Recursive Functions

- Functions that call themselves to solve a problem by breaking it down into smaller, similar subproblems.

```
function factorial(n) {
    if (n <= 1) return 1;
    return n * factorial(n - 1);
}
```

4 Higher Order Function

- Higher order functions are functions which take other function as a parameter or return a function as a value. The function passed as a parameter is called callback.

Callback

- A callback is a function which can be passed as parameter to other function. See the example below.

```
// A callback function, the name of the function could be any name
const callback = (n) => {
    return n ** 2;
  };

  // Function that takes another function as a callback
  function cube(callback, n) {
    return callback(n) * n;
  }

  console.log(cube(callback, 3)); // 27
```

Setting time

In JavaScript we can execute some activities in a certain interval of time or we can schedule(wait) for some time to execute some activities.

- setInterval
- setTimeout

Setting Interval using a setInterval function

- In JavaScript, we use setInterval higher order function to do some activity continuously with in some interval of time.
- The setInterval global method take a callback function and a duration as a parameter.
- The duration is in milliseconds and the callback will be always called in that interval of time

```
// syntax
function callback() {
  // code goes here
}
setInterval(callback, duration)

function sayHello() {
  console.log('Hello')
}
setInterval(sayHello, 1000) // it prints hello in every second, 1000ms is 1s
// ----------------------------------------------------------------------
```

Setting a time using a setTimeout

- In JavaScript, we use setTimeout higher order function to execute some action at some time in the future.
- The setTimeout global method take a callback function and a duration as a parameter.
- The duration is in milliseconds and the callback wait for that amount of time.

```
// syntax
function callback() {
  // code goes here
}
setTimeout(callback, duration) // duration in milliseconds

function sayHello() {
  console.log('Hello')
}
setTimeout(sayHello, 2000) // it prints hello after it waits for 2 seconds.
```

Tasks

1. Write a program to find the area of a triangle using function
2. Write a JavaScript function that accepts a number as a parameter and check the number is prime or not.
3. Write a JavaScript function that checks whether a passed string is palindrome or not?
4. Write an anonymous function that takes two numbers as arguments and returns their product.
5. Create an arrow function that squares a given number.
6. Write a recursive function in JavaScript to calculate the nth Fibonacci number. The Fibonacci sequence is defined as follows: the first two numbers are 0 and 1, and each subsequent number is the sum of the two preceding ones.

7. What is the difference between function parameters and arguments?

8. What is the purpose of the return statement in a function, and what does it do?

9. What are the differences between arrow functions and regular functions in JavaScript?

10. What is the difference between local and global scope in JavaScript functions?

11. Create a Function to Convert Celsius to Fahrenheit using arrow function.

12. Create a Function to print Greeting Message using arrow function.

Thank You